

Helix Thready — Coding Standards

Table of Contents

1. Language & toolchain
2. TDD — reproduce-first [§11.4.43]
3. Design patterns
4. SOLID & core principles
5. Decoupling [§11.4.28]
6. Concurrency
7. Error handling & observability
8. Anti-bluff discipline [§11.4.27]
9. Naming, layout, formatting
10. Non-Go surfaces

Rev	Date	Author	Change
1	2026-07-21	swarm (development)	Initial draft — Go 1.26, TDD, patterns, SOLID, decoupling, concurrency
2	2026-07-21	swarm (development, review)	Review pass — clarified operator precedence in the anti-bluff embedder guard
3	2026-07-22	swarm (development, pass 3)	Anchored the anti-bluff embedder guard to the VERIFIED <code>HelixLLM.NewEmbedder</code> source signature (the exact silent-fallback trap it defends against)
4	2026-07-22	swarm (development, critic pass)	Completeness fix — enumerated the canonical 15 mandated test types (authoritative final request §9.1) in §8 so every item's <code>test_types</code> field is verifiable against a self-contained reference; clarified sub-variants (mutation/contract/race/...) are refinements, not a 16th type

These are the engineering standards every `ATM-NNN` implementation must meet. They implement the final request §5.2 development principles and the Constitution's TDD [§11.4.43], decoupling [§11.4.28], and anti-bluff [§11.4.27] mandates.

Table of Contents

- 1. Language & toolchain
- 2. TDD — reproduce-first [§11.4.43]
- 3. Design patterns
- 4. SOLID & core principles
- 5. Decoupling [§11.4.28]
- 6. Concurrency
- 7. Error handling & observability
- 8. Anti-bluff discipline [§11.4.27]
- 9. Naming, layout, formatting
- 10. Non-Go surfaces

1. Language & toolchain

- **Primary language: Go 1.26.x** [CONSTITUTION silent → project pins latest stable]. The Constitution does not pin a Go version; the project pins the latest stable, matching org tooling (1.25/1.26). `go.mod` declares `go 1.26`.
- **Formatting/vetting (mandatory):** `gofmt -s`, `go vet`, `staticcheck`, `golanci-lint` (with `errcheck`, `govet`, `staticcheck`, `ineffassign`, `revive`). CI-equivalent enforcement runs in the local git hooks [§11.4.75/156] — there is no server CI.
- **Race + mutation:** `go test -race` on every package; `go-mutesting` mutation testing plus paired-mutation anti-bluff gates (§8).
- **No cgo unless required:** dev uses cgo-free `modernc.org/sqlite`; cgo is allowed only where a capability demands it (e.g. `gosseract` / Tesseract in the OCR adapter `ATM-033`), gated behind a build tag and documented.

2. TDD — reproduce-first [§11.4.43]

Every change is **reproduce-first**: write the failing (RED) test that reproduces the requirement or bug **before** any fix; the same test asserts the fix (GREEN); then extend to all cases. Iron Law [§11.4.102]: **no fix without root-cause investigation first** — no guess-and-retry, no “probably flaky” reruns without captured forensic evidence.



Explanation (for readers/models that cannot see the diagram). The first two nodes are the discipline’s hard order. A requirement or bug becomes a RED test that reproduces it precisely — never a fix attempt —

and only *after* a completed root-cause investigation (the §11.4.102 Iron Law forbids fixing before understanding) does the minimal implementation follow, turning the same test GREEN. The edge from RED to RC to IMPL is deliberately not a shortcut: you may not skip the investigation node and jump straight to a patch.

The tail of the flow is where a single passing test becomes releasable evidence. Once GREEN, the work extends to all edge cases and adds the applicable mandated test types (the 15-type set where relevant), and finally passes the independent Fable review [§11.4.209]. The RED test is the contract throughout: it must fail *for the right reason* before it is allowed to pass, which is what distinguishes a real reproduction from a test that would pass against a stub — the same anti-bluff property the paired-mutation gates (§8) enforce mechanically.

Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: `diagrams/tdd-reproduce-first.mmd`.

```
// reproduce-first: the RED test is written and MUST fail before the dispatcher exists.
func TestDispatcher_MultiHashtag_DownloadBeforeResearch(t *testing.T) {
    post := Post{Hashtags: []string{"#Video", "#Research"}}
    got := NewDispatcher(testGraph).Plan(post) // Plan() not yet implemented -> RED
    want := []SkillStep{{Skill: "video-download", Order: 10}, {Skill: "deep-research", Order: 40}}
    if !reflect.DeepEqual(got, want) {
        t.Fatalf("precedence violated: download must precede research\n got=%v\nwant=%v", got, want)
    }
}
```

3. Design patterns

The Constitution does not mandate specific GoF patterns; they are adopted as **project engineering standards** per final request §5.2. Standard patterns and where they apply:

Pattern	Thready usage
Adapter	<code>OCRProvider</code> (Tesseract/PaddleOCR), messenger channels (Telegram/Max), download sources (HTTP/FTP/SMB)
Factory / Factory-of-Factories	<code>VectorStore</code> (pgvector/Qdrant), embedding-provider selection, download-source construction
Strategy	Retry/back-off policies, transcode profiles, embedding-model choice
Circuit-Breaker	Guarding flapping external systems (present in <code>LLMProvider</code> / <code>filesystem</code> / <code>lets_encrypt</code>)
Observer	Event Bus subscriptions; processing progress events
Facade	The SDK surface over the REST/Protobuf core; the Asset Service over storage+filesystem
Mediator	The Skill-dispatch engine coordinating download/convert/analyze/research steps
Proxy	Asset links that resolve through the Asset Service, never direct file paths

```
// Adapter + Factory: a first-class OCRProvider seam (closes VisionEngine's no-OCR gap,
// ATM-033).
type OCRProvider interface {
    Recognize(ctx context.Context, img image.Image) (OCRResult, error) // per-word boxes + text
    Name() string
}
type OCRResult struct{ Text string; Words []WordBox }
type WordBox struct{ Text string; Rect image.Rectangle; Conf float64 }

func NewOCRProvider(cfg OCRConfig) (OCRProvider, error) { // Factory selects the concrete
    adapter
    switch cfg.Engine {
    case "tesseract": return newTesseractAdapter(cfg) // cgo gossertact or subprocess
    case "paddle":    return newPaddleAdapter(cfg)    // hard/multilingual scans
    default:          return nil, fmt.Errorf("ocr: unknown engine %q", cfg.Engine)
    }
}
```

4. SOLID & core principles

- **Single-Responsibility** — one reason to change per type; the dispatch engine dispatches, it does not embed nor transcode.
- **Open-Closed** — extend via new adapters/strategies, not by editing existing switch bodies; register new providers through the factory.
- **Liskov** — every `OCRProvider` / `VectorStore` / `Channel` implementation is substitutable; contract tests assert this (`ATM-064`).

- Interface-Segregation — small, purpose-built interfaces (`Recognize` , `OpenSeekable`), not god-interfaces.
- Dependency-Inversion — depend on interfaces, inject concretes via config (`§11.4.28`).
- Plus **DRY**, **KISS**, heavy decoupling and reuse. Prefer composition over inheritance.

5. Decoupling `§11.4.28`

VERIFIED at source (Constitution §11.4.28 / CONST-051). Rules for every Thready module:

- **Project-not-aware + config-injected**. A module never hardcodes Thready specifics; it takes a `Config` and dependencies via constructor injection.
- **Generic interfaces + abstract factories** for variants; no hard coupling to a concrete backend.
- **Dependency layout** `<root>/<name>/` or `<root>/submodules/<name>/`; **no nested own-org submodule chains** (depth-1 exception only for constitution-anchored engines with `helix-deps.yaml`).
- New capabilities become their **own repo** (see `build-new-subsystems.md`), never a vendored copy.

```
// Dependency inversion + config injection: the dispatcher depends on interfaces it does not
// construct.
type Dispatcher struct {
    graph SkillGraph      // read-only knowledge source (helix_skills)
    queue TaskQueue       // digital.vasic.background
    bus    EventPublisher // digital.vasic.eventbus
}
func NewDispatcher(cfg Config, graph SkillGraph, queue TaskQueue, bus EventPublisher)
    *Dispatcher {
    return &Dispatcher{graph: graph, queue: queue, bus: bus} // nothing project-specific
    hardcoded
}
```

6. Concurrency

Final request §5.2: “heavy use of atomic, non-blocking operations, correct data structures and synchronization.” Standards:

- Prefer **channels** and `context.Context` for cancellation/deadlines; every exported blocking call takes a `ctx`.
- Use `sync/atomic` and `sync.Once` / `sync.Map` where a mutex would be a bottleneck; document the invariant each atomic protects.
- **Idempotent single-claim** for the per-post processor (`ATM-023`) via Postgres advisory locks — the runtime analogue of the §11.4.176 exactly-once claim.

- Bound concurrency: the BackgroundTasks worker pool ([DEFAULT – adjustable] 32 workers), per-Skill concurrency caps, download concurrency delegated to Boba/MeTube/Download-Manager pools (Q4).
- Always `go test -race`; a data race is a release blocker.

```
// Non-blocking exactly-once claim: TryAcquire returns false if another worker already holds the post.
func (p *Processor) TryClaim(ctx context.Context, postID int64) (bool, error) {
    // pg_try_advisory_xact_lock is non-blocking: it returns immediately, no goroutine parks.
    var ok bool
    err := p.db.QueryRowContext(ctx, `SELECT pg_try_advisory_xact_lock($1)`, postID).Scan(&ok)
    return ok, err // ok==false => another worker owns this post; skip without blocking
}
```

7. Error handling & observability

- **Wrap, don't swallow:** `fmt.Errorf("dispatch %d: %w", id, err)`; never discard an error without a logged reason (`errcheck` enforces).
- **Typed sentinels** for control flow (`errors.Is` / `errors.As`); no string-matching on messages.
- **Structured logging** via `digital.vasic.observability` (logrus + correlation ids); OTel spans on every cross-service call; Prometheus metrics on queues, latencies, retries.
- **Honest SKIP** [§11.4.3/52]: when a capability is genuinely unavailable (e.g. `pandoc` missing for Docs Chain, credentials absent), SKIP with a logged reason — never fake success.
- **Never log secrets** [§11.4.10]; credentials are runtime-load-only and redacted in logs.

8. Anti-bluff discipline [§11.4.27]

The 15 mandated test types (canonical, VERIFIED — final request §9.1 / [§11.4.27]). Every `ATM-NNN` names the *applicable subset* of these; the target is **100% test-type coverage per feature × platform cell** (every applicable type present), not a line-percentage. The testing area (`../testing/index.md`) expands each into runnable banks; this is the reference list every item's `test_types` field is checked against:

#	Type	Notes
1	unit	the only tier where mocks/stubs/TODO are permitted [§11.4.27]
2	integration	real cross-module contracts; no fakes beyond unit
3	e2e	full user-journey against the real system
4	full-automation	end-to-end automated flows (no manual steps)
5	security	authn/authz, secret-leak, fuzzing, dependency-CVE, attack sims
6	DDoS	flood/abuse simulation
7	scaling	horizontal/fan-out behavior under growth
8	chaos	fault injection (container/broker/transfer kill → recover)
9	stress	load beyond nominal to find the breaking point
10	performance	SLO validation (API p95 < 150 ms, search < 500 ms)
11	benchmarking	quantified throughput/latency baselines + regression tracking
12	UI	rendering/interaction correctness (client surfaces)
13	UX	usability/accuracy (e.g. OCR transcription fidelity, a11y)
14	Challenges	vasic-digital/challenges per-feature real-use-case scenario banks
15	HelixQA	HelixDevelopment/helix_qa YAML banks with mandatory runtime evidence (screenshots/logcat/video/stacktrace) — the org anti-bluff rule

Sub-variants an item may additionally cite — [mutation](#) / [paired-mutation](#) (§8 below), [contract](#) (Liskov substitutability, [ATM-064](#)), [race](#) ([go test -race](#)), [migration round-trip](#) (expand-contract, §10), [penetration](#) / [fuzzing](#) (rolled under **security**), [visual-regression](#) (Panoptic/[VisualRegression](#) / [ScreenDiff](#), rolled under **UI**) — are refinements of the 15, not a 16th category. An item that omits an *applicable* type is incomplete, not merely thin.

A green test must prove **real behavior**, not a stub. This is the single most important standard for Thready because the gap register flags several dependencies as scaffolds:

- **No mocks beyond unit tests.** Integration/e2e/system/security/performance/etc. exercise the real system [\[§11.4.27\]](#). Mocks/stubs/TODOs are unit-only.
- **Paired-mutation gates** [\[CONST-035\]](#) : for any module the gap register flags SCAFFOLD/DESIGN-ONLY (HelixLLM embedder, TOON, session_orchestrator, Security-KMP), add a gate that mutates the

implementation and asserts the test **FAILs**, then restores and asserts it **PASSes** — proving the test detects real behavior. `ATM-058` runs this sweep before reliance.

- **Never claim a stub works.** If `HELIX_EMBEDDING_PROVIDER` is the `HashEmbedder`, the code **fails loudly** in a RAG/search context (`ATM-040`), it does not warn-and-continue.

The exact trap this guard defends against `[VERIFIED-SOURCE]` . Read at source — `HelixLLM/internal/knowledge/embedding_providers.go`: `func NewEmbedder(provider, apiKey, model string, dimension int) (Embedder, error) maps "llama" → the real LlamaEmbedder, but "hash", "local" and any unrecognised provider name → NewHashEmbedder(dimension) — a deterministic non-semantic stub, today with only a startup WARNING (cmd/helixllm/embedder_warning_test.go). A typo in HELIX_EMBEDDING_PROVIDER therefore silently yields garbage relevance. The RequireSemanticEmbedder guard below rejects hash / local / unknown in any semantic context, and ATM-040 converts the WARNING into a hard fail; a paired-mutation gate proves the guard actually fires.`

```
// Anti-bluff guard: refuse the non-semantic hash embedder in any semantic context (closes GAP 2.1).
func RequireSemanticEmbedder(cfg EmbedConfig) error {
    // Explicit parens: reject the hash provider outright, OR a "local" provider with no model loaded
    // (the HashEmbedder trap). Parenthesized so precedence is unmistakable, not relying on && > ||.
    if cfg.Provider == "hash" || (cfg.Provider == "local" && cfg.Model == "") {
        return fmt.Errorf("refusing HashEmbedder in semantic context: set HELIX_EMBEDDING_PROVIDER=llama")
    }
    return nil
}
```

9. Naming, layout, formatting

- **Lowercase snake_case** for repo/dir/file names `[§11.4.29 / CONST-052]` ; Go identifiers follow standard Go casing.
- Package layout: `cmd/` (entrypoints), `pkg/` (public), `internal/` (private), `test/` (fixtures); one responsibility per package.
- Every exported symbol has a doc comment; every package has a `doc.go` or package comment.
- Every `.md` gets HTML/PDF/DOCX siblings via Docs Chain `[§11.4.65]` .

10. Non-Go surfaces

Surface	Standard
TypeScript/Angular	Angular 19 (product) / 22 (marketing) on <code>design_system</code> ; Jasmine+Karma unit, Cypress e2e (<code>cypress-axe</code>), Prettier+ESLint
Rust (Tauri)	<code>cargo fmt</code> , <code>cargo clippy -D warnings</code> , <code>cargo test</code>
Kotlin/KMP	ktlint/detekt; Kotest/kotlin-test; JUnit + Compose instrumented tests
Swift	swiftformat/swiftlint; XCTest
SQL	PostgreSQL DDL; migrations up/down via <code>migration.Runner</code> ; expand-contract
YAML/Proto	OpenAPI 3.1 + Protobuf via the <code>helix_proto</code> pattern (<code>buf</code> , <code>openapi-generator</code>)

Made with love ♥ by Helix Development.